

Qwen-Audio-Agent Technical Report

Alibaba Token Foundry, Alibaba Group

Abstract

We present Qwen-Audio-Agent, a harness that combines full-duplex voice interaction with asynchronous task execution through a foreground-background architecture. A Frontend Agent manages dialogue and selects between direct tool use and delegation, while a Backend Agent carries out delegated tasks in a separate context. An Orchestration Runtime maintains task state, coordinates requests for user input and authorization, and schedules the return of results to the conversation. The runtime separates speech interruption from task cancellation and execution completion from result delivery, allowing conversation to continue while delegated work proceeds. Environmental events and persistent memory provide context within and across sessions. Independent adapters support integration with different frontend models, backend agents, and clients. We instantiate the architecture in desktop assistance, intelligent cockpits, and voice customer service. On an in-house cockpit benchmark of 134 cases, mixed execution achieves a task success rate of 91.04%, compared with 72.39% and 80.60% for the direct and all delegated configurations, respectively. In a separate latency evaluation on matched successful turns, mixed execution reduces mean task execution latency by 26.73% and 30.91% relative to these baselines, respectively. These results support the complementary use of direct tool calls for immediate operations and backend delegation for multi-step tasks.

1 Introduction

Voice agents increasingly perform tasks that span several conversational turns. A user may revise a request while a search is in progress, provide a missing constraint during a transaction, or ask about a document that is still being processed. The agent must preserve task state across these exchanges while responding to new input. For full-duplex voice agents, conversation, task execution, and result delivery must therefore be coordinated across different timescales.

Coordinating these activities requires separate control of conversation and task execution. Interrupting a spoken response should not implicitly cancel ongoing work. Similarly, a task may complete while the user is speaking, requiring its result to be retained until an appropriate point in the conversation. We use *harness* to denote the runtime infrastructure that connects models, tools, and execution environments and implements these interaction and task-management mechanisms.

Research on spoken turn-taking examines response timing and overlapping speech (Skantze, 2021), while full-duplex models and benchmarks address simultaneous listening and speaking (Défossez et al., 2024; Lin et al., 2025). Complementary work on language agents studies reasoning with environmental feedback (Yao et al., 2023), multi-agent conversation (Wu et al., 2024), and parallel tool execution (Kim et al., 2024).

Recent systems explore related forms of asynchronous voice interaction. OpenAI’s gpt-realtime supports asynchronous function calls (Bakkum, 2025), while GPT-Live delegates reasoning and tool execution from a voice frontend to a backend (Healy et al., 2026; OpenAI, n.d.). Thinking Machines Lab’s Interaction Models pair an interaction model with an asynchronous background model (Thinking Machines Lab, 2026). Gemini Live supports asynchronous function calls and, through its Extended Thinking model, background reasoning and tool use during spoken interaction (Google, 2026a,b). These developments motivate a common coordination layer for integrating different frontend models and backend execution systems.

Qwen-Audio-Agent addresses this need through a common runtime for coordinating voice interaction and delegated tasks. The key design choice is to manage tasks independently of

conversational turns, allowing users to continue the dialogue while earlier work proceeds. The runtime maintains task state and coordinates result delivery, while separate adapters support different frontend models, backend agents, and clients. Environmental events and persistent memory maintain context within and across sessions. The implementation is released as open source¹.

Table 1 compares the capabilities of Qwen-Audio-Agent and related voice systems.

Table 1: Capability comparison of voice agent systems.

Capability	gpt-realtime	GPT-Live	Gemini Live	Qwen-Audio-Agent
Direct frontend tool use	✓	×	✓	✓
Dialogue during execution	✓	✓	✓	✓
Cross-provider frontend support	×	×	×	✓
Extensible backend agents	△	✓	△	✓
Backend task lifecycle management	△	△	△	✓
Extensible environmental events	△	△	△	✓
Persistent cross-session memory	△	△	△	✓
Open-source orchestration runtime	×	×	×	✓

✓: provided; △: requires application integration; ×: not exposed by the interface.

The report details task coordination, environmental events, and memory management, then examines their use in three application domains. A cockpit benchmark compares direct, all delegated, and mixed configurations to assess the effects of execution routing on task success and latency.

2 System Architecture

2.1 Foreground-background architecture

Qwen-Audio-Agent separates ongoing conversation from delegated task execution (Figure 1). The Frontend Agent interprets streaming input, maintains full-duplex dialogue, and chooses between direct tool calls and backend delegation. The Backend Agent carries out delegated tasks using its own context, tools, and workspace. The Orchestration Runtime tracks these tasks and routes progress updates, user input, and results between the two agents. Clients capture input, play speech, display artifacts, and perform supported local actions.

The frontend uses direct tools for immediate operations and delegates tasks that require extended reasoning or a separate execution environment. It selects an execution path according to the request and available capabilities. Both paths remain available within a session, so immediate requests can be handled while delegated work continues.

Independent adapters allow frontend models and backend agents to be integrated through a common set of interfaces. Frontend adapters translate streaming events and tool calls, while backend adapters expose task submission, progress, results, and supported controls. Backend adapters use Agent Client Protocol (ACP) (Agent Client Protocol, n.d.), Agent2Agent (A2A) (A2A Project, 2026), or custom interfaces. Each adapter declares the optional controls it supports, such as cancellation and requests for additional user input.

¹Source code: <https://github.com/QwenAudio/qwen-audio-agent>.

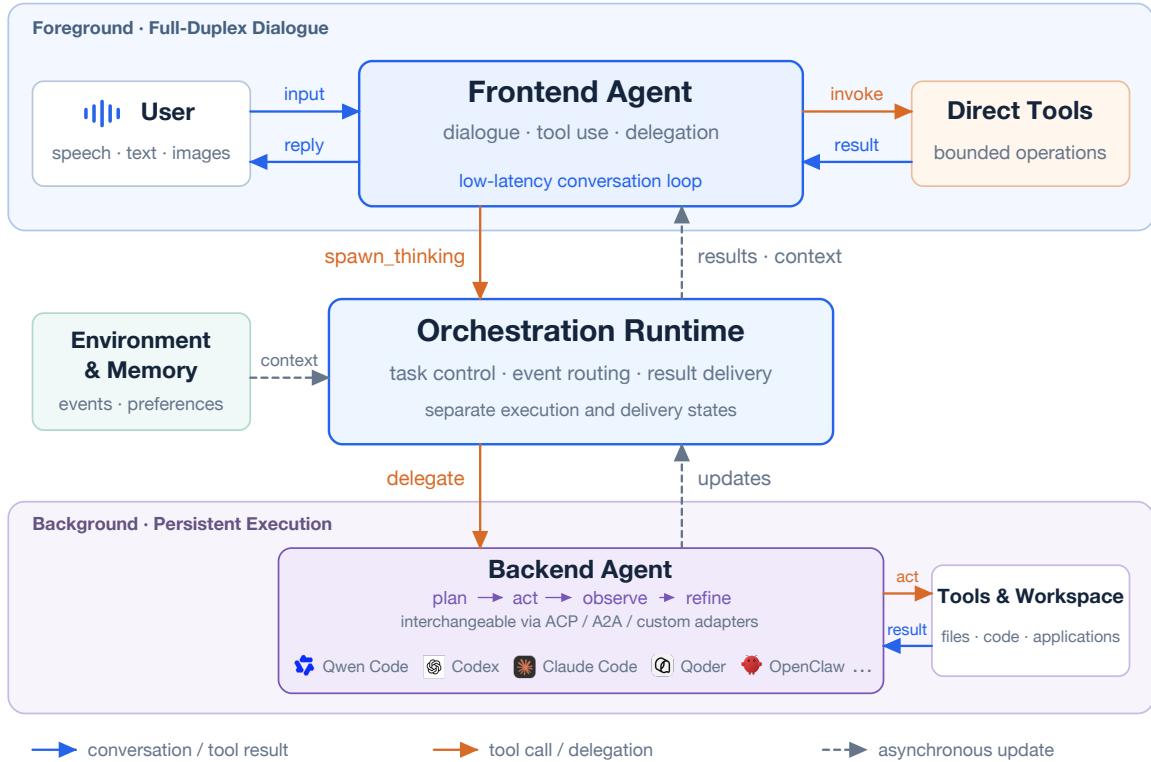


Figure 1: Foreground-background architecture of Qwen-Audio-Agent. The Orchestration Runtime coordinates full-duplex dialogue and asynchronous task execution. The backend examples illustrate interchangeable agent implementations.

2.2 Asynchronous task coordination

Asynchronous delegation is exposed through the `spawn_thinking` tool call. The frontend submits a self-contained objective that includes the relevant constraints and input references. This objective provides the context needed for execution, since frontend dialogue history and memory are not automatically transferred to the backend. The runtime creates a task record and immediately acknowledges acceptance with a task identifier. The frontend can then continue the conversation while the task is queued or running. Figure 2 illustrates how this task persists across subsequent conversational turns.

Each task record contains its objective, status, progress, outputs, and pending requests. The common lifecycle distinguishes queued and running states from the terminal states completed, failed, and cancelled. The frontend can query the same record in later turns. When supported by the backend, requests for information or authorization remain linked to the originating task, allowing the runtime to route the user’s response to the corresponding pending request. Follow-up work can be submitted as a new task that refers to an earlier result.

Speech interruption and task cancellation are controlled independently. Interrupting a response stops spoken output while preserving the task’s execution state. The runtime forwards task cancellation requests through the controls supported by the backend. This distinction allows the conversation to be redirected without implicitly terminating unfinished work.

Result delivery is scheduled separately from execution completion. The runtime retains completed results while the user is speaking or a frontend response is in progress. Results that arrive within the same collection window are combined into one response request. The frontend presents results in the current dialogue context, while artifacts remain linked to their tasks.

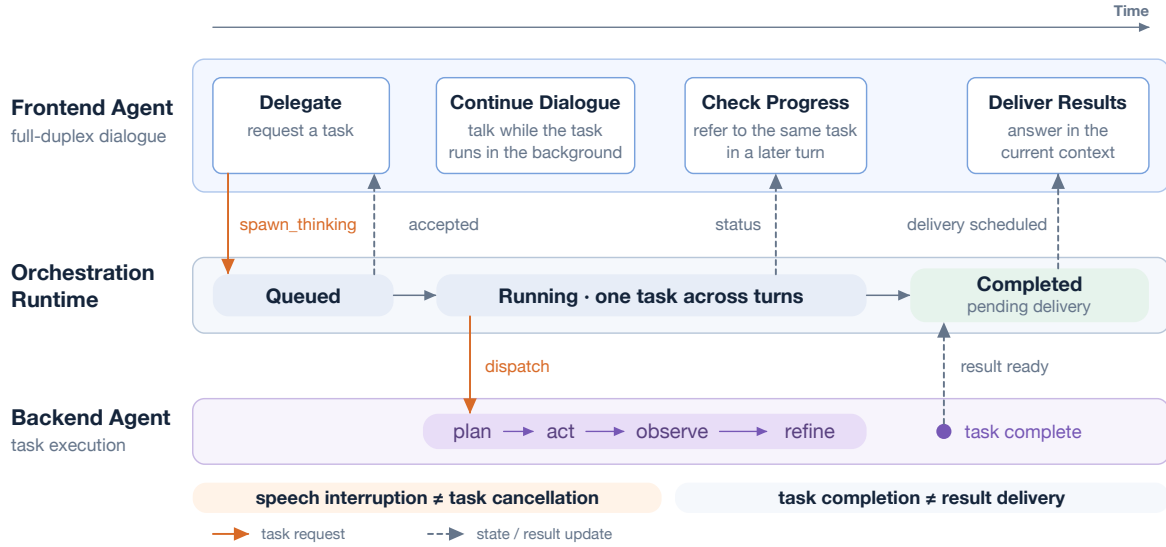


Figure 2: Illustrative timeline of a delegated task. The Frontend Agent continues dialogue during execution, while the Orchestration Runtime tracks task state and schedules the completed result for delivery.

Separate execution and delivery records allow an unsuccessful delivery to be retried without repeating completed work.

2.3 Environment perception and unified events

Environmental context can change independently of spoken input. Supported clients supply visual observations and application or device state, enabling the frontend to interpret subsequent requests against the current environment. For example, when a user changes the destination through the cockpit interface, the new destination can be added to conversational context without triggering a spoken response. Clients also report whether requested actions succeeded, providing execution feedback to the runtime.

IrisTK coordinates multimodal modules through standardized events (Skantze and Johansson, 2015). In Qwen-Audio-Agent, application and device state changes use a unified event protocol, independent of the frontend provider. Each event type defines a schema, retention rule, and response policy. The runtime validates an event and either handles it directly, updates frontend context silently, schedules a response, or interrupts the current response and requests a new one. Frontend adapters translate these actions into provider-specific messages.

2.4 Memory and user context

Prior work explores memory retrieval and reflection (Park et al., 2023) and hierarchical memory management (Packer et al., 2023). LongMemEval evaluates the retention and updating of information across sessions (Wu et al., 2025). In Qwen-Audio-Agent, session context combines core interaction rules, an assistant profile, explicit user preferences, and long-term user memory. The frontend uses a bounded memory snapshot to construct its context, while remote retrieval and consolidation can proceed asynchronously. Task records and reference documents remain separate from personal memory.

The default memory implementation supports explicit edits during dialogue and extracts additional long-term instructions and stable facts at session boundaries. An optional learning process adds inferred preferences only when supported by consistent evidence across sessions. Current user requests take precedence over stored preferences, and explicit preferences take

precedence over inferred preferences. Revision checks prevent delayed background updates from overwriting newer corrections. Preferences rejected by the user cannot be added again automatically. These mechanisms adapt the context of later interactions without changing model parameters.

3 Applications

We instantiate Qwen-Audio-Agent in desktop assistance, intelligent cockpits, and voice customer service. These applications place different demands on task coordination: iterative work on files, interaction with a changing environment, and transactions governed by policy and user approval.

3.1 Desktop assistance

Office and programming tasks often require inspecting a workspace, planning a sequence of actions, and revising the plan based on execution feedback (Xie et al., 2024; Yang et al., 2024). The desktop implementation delegates these objectives to a backend agent with workspace tools, allowing document editing, code execution, and verification to proceed across conversational turns. During execution, the frontend remains available for dialogue and progress queries. Completed tasks return summaries and artifact references, while the full deliverables remain accessible through the workspace or client. Users can inspect and revise these artifacts through subsequent dialogue, linking spoken delegation to verifiable outputs.

3.2 Intelligent cockpits

Cockpit interaction combines immediate control requests with longer tasks, while vehicle and application state can change through tool calls or direct interface actions. We address this requirement through a shared domain service: frontend tools, backend tools, and the graphical interface use the same business state and execution logic. Relevant state changes are returned as environmental events, keeping subsequent dialogue grounded in the current vehicle and application context.

The reference application assigns tools by domain: immediate controls and navigation to the frontend, and selected shopping and research tasks to the backend. Vehicle controls remain available while background work proceeds. In the benchmark, mixed execution instead selects adaptively between two paths with access to the same tools.

3.3 Voice customer service

Customer-service workflows require policy interpretation, access to business records, and user decisions at specific execution steps (Yao et al., 2025). The retail and airline implementations combine policy retrieval with a shared business service whose tools and decision tables apply eligibility rules and calculate transaction amounts. Operations requiring approval follow a preview-and-commit sequence. The tool first prepares the proposed change, and the backend task pauses while the frontend obtains the user’s decision. If the user approves, the task resumes and commits the operation. Linking this decision to the pending task preserves the execution context across conversational turns.

In addition to these three domains, Qwen-Audio-Agent has also been deployed in digital-human and intelligent-hardware applications.

4 Experimental Evaluation

4.1 Experimental setup

We compare four execution configurations on an in-house cockpit benchmark containing 134 cases: 86 short interactions and 48 multi-step tasks, spanning 159 user turns. All configurations use the same 41 business tools, initial states, and evaluation criteria. Qwen Audio 3.0 Realtime Plus serves as the Frontend Agent, with qwen3.8-max as the Backend Agent. The three audio configurations receive identical inputs and differ in execution routing: direct uses frontend tool calls, all delegated relies on the backend for all business-tool use, and mixed execution adaptively selects between the two paths. A text-only backend baseline receives the corresponding text and executes tools directly.

Evaluation combines adapted FDB V3 metrics (Lin et al., 2026) with execution-based task success, consistent with the emphasis on task outcomes in interactive agent benchmarks (Yao et al., 2025; Ray et al., 2026).

4.2 Task completion

Table 2: Results on the in-house cockpit benchmark. The upper panel reports capability on 134 cockpit cases: F1 uses a 0–1 scale, and the remaining metrics are percentages. The lower panel reports mean latency on a separate set of turns completed successfully by all configurations. Bold denotes the best value in each column.

Configuration	Capability evaluation			
	Tool F1	Argument accuracy	Task success	Reply quality
Audio frontend, direct tools	0.947	93.6	72.39	95.5
Audio frontend, all delegated	0.908	89.8	80.60	89.6
Text backend, direct tools	0.963	94.1	90.30	100.0
Audio frontend, mixed execution	0.962	95.2	91.04	97.8

Configuration	Mean task execution latency (s) ↓		
	Short tasks (74 turns)	Multi-step tasks (6 turns)	Overall (80 turns)
Audio frontend, direct tools	1.488	67.705	6.455
Audio frontend, all delegated	4.266	38.653	6.845
Text backend, direct tools	3.149	35.449	5.572
Audio frontend, mixed execution	1.591	43.430	4.729

Table 2 shows that mixed execution achieves the highest task success rate at 91.04%. Direct frontend execution succeeds on 83 of 86 short cases but only 14 of 48 multi-step cases. The all delegated configuration improves multi-step completion to 37 cases, while short-case completion falls to 71. Mixed execution completes 84 short and 38 multi-step cases, combining strong performance on immediate operations with the backend’s ability to execute longer action sequences.

4.3 Task execution latency

Latency is measured separately on 80 user turns from 71 cases, with all four configurations completing each turn successfully. Measurement begins at the end of the spoken request, or at text submission, and ends at task completion; subsequent reply generation and playback are excluded.

Mixed execution achieves the lowest overall mean latency at 4.729 s, a reduction of 26.73% relative to direct frontend execution and 30.91% relative to the all delegated configuration

(Table 2). On these turns, mixed execution routes all short tasks to the frontend and all multi-step tasks to the backend. It retains low latency on short requests while reducing multi-step latency relative to direct frontend execution. The all delegated configuration and the text-only backend baseline are faster on multi-step turns.

5 Conclusion

Qwen-Audio-Agent extends voice interaction to tasks whose execution continues across conversational turns. Explicit task management and scheduled result delivery allow ongoing dialogue and background work to coexist within the same interaction. Independent integration interfaces make this architecture applicable to different models, backend agents, and application domains. The cockpit evaluation supports retaining both execution paths: mixed execution achieves higher task success and, on matched successful turns, lower overall mean execution latency than the direct and all delegated configurations. Future work will examine conversational behavior during task execution, recovery over longer sessions, and personalization across a broader range of models, agents, and devices.

6 Contributions and Acknowledgments

All authors of Qwen-Audio-Agent are listed in alphabetical order by last name.

Authors: Chong Deng, Yunjie Ji, Yuxiang Kong, Xiangang Li, Xu Li, Binbin Zhang, Haina Zhu, and Jianheng Zhuo.

References

- A2A Project. Agent2Agent (A2A) protocol specification. Protocol specification, 2026. URL <https://a2a-protocol.org/v1.0.0/specification/>. Version 1.0.0. Accessed: 2026-09-21.
- Agent Client Protocol. Agent Client Protocol: Overview. Protocol documentation, n.d. URL <https://agentclientprotocol.com/protocol/v1/overview>. Accessed: 2026-09-21.
- Peter Bakkum. Developer notes on the Realtime API. OpenAI Developers, 2025. URL <https://developers.openai.com/blog/realtime-api>. September 12, 2025.
- Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: A speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024. URL <https://arxiv.org/abs/2410.00037>.
- Google. Gemini 3.8 Live. Google AI for Developers, 2026a. URL <https://ai.google.dev/gemini-api/docs/models/gemini-3.8-live>. Last updated September 15, 2026. Accessed: 2026-09-21.
- Google. Thinking in the Live API. Google AI for Developers, 2026b. URL <https://ai.google.dev/gemini-api/docs/live-api/thinking>. Last updated September 15, 2026. Accessed: 2026-09-21.
- Kait Healy, Karsten Schroer, and Erika Kettleson. GPT-Live evaluation guide. OpenAI Cookbook, 2026. URL https://developers.openai.com/cookbook/examples/audio/voice_agent_evaluation. September 10, 2026.
- Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W. Mahoney, Kurt Keutzer, and Amir Gholami. An LLM compiler for parallel function calling. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 24370–24391. PMLR, 2024. URL <https://proceedings.mlr.press/v235/kim24y.html>.
- Guan-Ting Lin, Jiachen Lian, Tingle Li, Qirui Wang, Gopala Anumanchipalli, Alexander H. Liu, and Hung-yi Lee. Full-Duplex-Bench: A benchmark to evaluate full-duplex spoken dialogue models on turn-taking capabilities. In *2025 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2025. URL <https://arxiv.org/abs/2503.04721>.
- Guan-Ting Lin, Chen Chen, Zhehuai Chen, and Hung-yi Lee. Full-Duplex-Bench-v3: Benchmarking tool use for full-duplex voice agents under real-world disfluency. *arXiv preprint arXiv:2604.04847*, 2026. URL <https://arxiv.org/abs/2604.04847>.
- OpenAI. Delegation and tools in GPT-Live. OpenAI API documentation, n.d. URL <https://developers.openai.com/api/docs/guides/live-delegation>. Accessed: 2026-09-21.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023. URL <https://arxiv.org/abs/2310.08560>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, pages 1–22. Association for Computing Machinery, 2023. doi: 10.1145/3586183.3606763. URL <https://doi.org/10.1145/3586183.3606763>.
- Soham Ray, Keshav Dhandhan, Victor Barres, and Karthik Narasimhan. τ -Voice: Benchmarking full-duplex voice agents on real-world domains. *arXiv preprint arXiv:2603.13686*, 2026. doi: 10.48550/arXiv.2603.13686. URL <https://arxiv.org/abs/2603.13686>.
- Gabriel Skantze. Turn-taking in conversational systems and human-robot interaction: A review. *Computer Speech & Language*, 67:101178, 2021. doi: 10.1016/j.csl.2020.101178. URL <https://doi.org/10.1016/j.csl.2020.101178>.
- Gabriel Skantze and Martin Johansson. Modelling situated human-robot interaction using IrisTK. In *Proceedings of the 16th Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 165–167. Association for Computational Linguistics, 2015. doi: 10.18653/v1/W15-4624. URL <https://aclanthology.org/W15-4624/>.
- Thinking Machines Lab. Interaction Models: A scalable approach to human-AI collaboration. *Thinking Machines Lab: Connectionism*, May 2026. doi: 10.64434/tml.20260511. URL <https://thinkingmachines.ai/blog/interaction-models/>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=pZiyCaVuti>.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024. URL <https://openreview.net/forum?id=BAakY1hNKS>.

- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, volume 37, pages 52040–52094. Curran Associates, Inc., 2024. doi: 10.52202/079017-1650. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5d413e48f84dc61244b6be550f1cd8f5-Abstract-Datasets_and_Benchmarks_Track.html.
- John Yang, Carlos Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, pages 50528–50652. Curran Associates, Inc., 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2210.03629>.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. URL <https://openreview.net/forum?id=roNSXZpUDN>.